

Laborator

Editoare de text, Gcc si Makefile

Curs: Sisteme de Operare

Cuprins

1. Editoare de Text.....	3
1.1. Introducere.....	3
1.2. Editorul vim	3
1.2.1. Prezentare functii	3
1.2.2. Moduri de functionare	4
1.2.3. Descriere detaliata moduri.....	4
1.2.4. Taste si comenzi utile	4
1.3. Alte editoare de text.....	6
2. Compilare si Rulare Programe C	7
2.1. Introducere.....	7
2.2. Instalare pachete compilator gcc/g++	7
3.3. Compilare cu gcc/g++ si rulare.....	8
3.4. Makefile	8
3.4.1. Introducere	8
3.4.2. Formatul unui Makefile.....	8
3. Exercitii de laborator.....	10

Pachete Debian/Ubuntu ce trebuie instalate: **emacs21, gcc, g++, libc6-dev, binutils, file, make**

Important!

Atunci cand folositi LiveCD s-ar putea sa mearga foarte incet instalarea pachetelor. Cauza este lipsa de memorie RAM disponibila. Astfel, se recomanda ca inainte de instalare sa inchideti mediul grafic cu una dintre comenzile:

```
$ sudo killall kdm
pentru KDM (Kubuntu) sau
$ sudo killall gdm
pentru Gnome (Ubuntu)
```

1. Editoare de Text

1.1. Introducere

Sistemele de operare moderne includ interfata grafica cu utilizatorul. Aceasta permite existenta unor editoare de text specializate, cum ar fi OpenOffice Writer si Microsoft Word pentru editare de documente profesionale, Kate, Programmer's Notepad si editorul din Visual Studio pentru scrierea codului sursa. Aceste editoare sunt de cele mai multe ori foarte simplu de utilizat, cu taste standard, avand functii avansate care pot fi invatate pe masura ce devin necesare.

Contrar, linia de comanda include editoare care cer mult mai mult timp pentru invatare si acomodare. Totusi, ca viitori programatori si depanatori va trebui sa lucrati la fel de usor si in linia de comanda si in interfata grafica.

Pentru editarea in linie de comanda sunt disponibile mai multe editoare, dintre care si **vim**. Vim este o varianta imbunatatita de vi, un editor de text pentru Unix aparut in 1976. Avantajele acestuia sunt dimensiunea mica si viteza mare fata de principalul sau concurent **Emacs**. Emacs a pornit de la un editor de text dar extensibilitatea acestuia a permis sa se ajunga la o aplicatie complexa care include chiar si un modul de Web Browser. In acest laborator vom studia numai vim, lasandu-va studiul Emacs ca o cercetare individuala.

Apoi, pentru a demonstra utilitatea editarii in linie de comanda, vom folosi vim pentru a crea un program C pe care apoi il vom compila. Dupa ce am verificat functionalitatea acestuia, vom automatiza sarcinile de compilare, de instalare si de deinstalare a programului pe baza unui Makefile.

1.2. Editorul vim

1.2.1. Prezentare functii

Editorul **vim** pune la dispozitie o multime de functii utile programatorilor:

- **syntax highlighting** pentru aproape toate limbajele existente – se subliniaza cuvintele cheie sau structurile predefinite din limbajul utilizat (C/C++, Java, Perl, etc. si Makefile)
- **autocomplete** ajuta la scrierea textului mai repede prin completarea cuvantului pe baza primelor litere (pe baza unui dictionar)
- **macro-uri** – posibilitatea de a inregistra o serie de actiuni pentru a fi repetate simultan. De exemplu, structurile mari de genul *for(...;...;...) { ... }* pot fi inregistrate si introduse automat cu ajutorul unei combinatii de taste
- **lansarea de comenzi shell direct din aplicatie** – de exemplu compilarea programului la care scrieti in acel moment pentru a verifica daca sintaxa este corecta sau chiar lansarea acestuia in functiune din interfata VIM fara a salva toate fisierele deschise
- **lucrul cu mai multe fisiere simultan** – se pot deschide mai multe ferestre cu fisiere diferite (foarte util programatorilor)

Pornirea editorului vim se face cu comanda **vim fisier**.

```
$ vim /etc/fstab
```

1.2.2. Moduri de functionare

Editorul vim foloseste mai multe moduri de functionare. Cele mai importante sunt:

- **NORMAL (Ecran)** – modul in care porneste vim. Folosit pentru navigare si pentru executarea de comenzi
- **INSERT** – folosit pentru inserarea de caractere in fisier
- **REPLACE** – asemanator modului INSERT, insa rescrie caracterele in loc sa le introduca in text
- **COMMAND** – permite scrierea de comenzi vim si rularea comenzilor shell din interiorul editorului
- **VISUAL** – este folosit pentru selectarea la nivel de caracter (copy/paste pe portiuni de text).

1.2.3. Descriere detaliata moduri

Modul **Normal**. In acest mod se pot face ajustari la textul deja existent. De cele mai multe ori acesta este utilizat pentru a parcurge un text deja scris. Exista optiuni de stergere, modificare, copiere, alipire (paste) si refacere (undo). Acesta este modul cel mai des folosit. **Pentru a reveni din orice mod in modul Normal se foloseste tasta <ESC>**. In acest mod tastele alfanumerice nu au rol de introducere de caractere ci de comenzi. De cele mai multe ori trecerea dintr-un mod in altul se face prin iesirea in modul **Normal** si apoi prin intrarea in noul mod.

Modul **Insert**. Acest mod se foloseste pentru introducerea de text in mod brut. Toate tastele alfanumerice se introduc direct in text, spre deosebire de modul Normal. Pentru a intra in modul Insert se foloseste tasta <i> in modul Normal. Dupa introducerea textului dorit, se revine in modul Normal cu tasta <ESC>. Exista mai multe variatii pe baza acestui mod, dintre care cea mai importanta este modul **Replace**. Acest mod permite inlocuirea caracterelor din textul deja introdus. Intrarea in modul Replace se face cu tasta <R> (shift-r) din modul Normal. Alternativ, se poate face comutarea intre Replace si Insert ajutorul tastei <Insert>.

Modul **Command** ajuta la introducerea de comenzi. De aici se pot porni si comenzile vim si comenzile shell. Pentru a intra in modul Command se introduce ":" sau "/" de la tastatura in modul Normal. In acel moment cursorul se muta in partea de jos a ecranului si VIM asteapta introducerea unei comenzi si apasarea tastei Enter.

Modul **Visual** permite copierea, stergerea si alipirea textului (copy=yank, cut, paste). Pentru a intra in acest mod se foloseste tasta <v> in modul Normal. Odata intrat in modul Visual, vim permite selectia textului care urmeaza a fi copiat/sters.

1.2.4. Taste si comenzi utile

Taste	Actiune (Modul Normal)
h/j/k/l sau sageti	Miscare cursor
Ctrl-f sau PgDown	Merge la pagina urmatoare
Ctrl-b sau PgUp	Merge la pagina anterioara
w	Avansaeza cu un cuvnt.

5w	Avanseaza cu x cuvinte. x=2,3,4 ...
b 5b	Se intoarce cu un cuvint. Se intoarce cu x cuvinte.
0 (zero) sau Home	Merge la inceputul liniei.
\$ sau End	Merge la sfarstiul liniei.
x Shift-g	Merge la linia x, sau la sfarsitul fisierului daca nu se specifica x. (exemplu 10 [Shift-G] merge la linia 10)
u	Anuleaza ultima schimbare. (Undo)
U	Anuleaza schimbarile facute pe linia curenta (Undo Line)
o i, a, A, Insert	Introduce o linie goala. Intra in modul Insert.
Ctrl-G	Afisaza pe ultima linie numele fisierului editat si numarul liniei pe care se afla cursorul.
Comenzi compuse	
[numar x] d <deplasare>	Sterge o zona de x ori. Daca nu se specifica x se considera x=1. Exemple: 2 d w – va sterge urmatoarele 2 cuvinte d \$ - sterge tot pana la sfarsitul liniei special: d d – sterge linia curenta
[numar x] c <deplasare>	Inlocuieste o zona. (O sterge si apoi merge in modul insert)
Comenzi fisiere (Modul Command)	
:e <nume fisier>	Incarca un fisier.
:w	Salveaza schimbarile in fisierul curent.
:w <nume fisier>	Salveaza schimbarile intr-un fisier.
:q	Iese din VIM.
!	Forteaza comanda. Ex: :q! - iese chiar daca nu sunt fiserele salvate :w! - daca nu exista fisierul, il creaza si apoi scrie in el.
Comenzi cautare (Modul Command)	
/ <cuvant>	Cauta cuvantul dat si pozitioneaza cursorul la prima aparitie, in modul Normal.
n	(In modul normal) Merge cu cursorul la urmatoarea aparitie a cuvantului cautat.
:%s/<cuvant vechi>/<cuvant nou>/g	Inlocuieste toate aparitiile cuvantului vechi cu cuvantul nou.
Comenzi selectie (Modul Visual)	
y	Copiaza zona selectata (copy=yank)
d	Sterge zona selectata si o tine in memorie (cut)
p	Introduce (alipeste) ultima zona copiata sau stearsa (paste)
Comenzi selectie (Modul Visual)	

%	(In modul normal, pozitionanad cursorul pe o paranteza) Duce cursorul la paranteza corespunzatoare.
:syntax on	Pornește Syntax Highlighting
Comenzi ferestre (Modul Command)	
:split	Imparte ecranul in doua ferestre separate orizontale
:vsplit	Imparte ecranul in doua ferestre separate verticale
Ctrl-W	Pentru a naviga între ferestre
:close sau :quit	Inchide fereastra curentă
:only	Inchide toate ferestrele mai puțin cea curentă

1.3. Alte editoare de text

În afara de vim, în linia de comandă Linux mai sunt disponibile și alte două editoare, Nano și Emacs. Nano poate fi pornit cu comanda nano. Acesta este un editor simplu dar cu care puteți avea probleme la liniile mai lungi de 80 de caractere. Emacs, datorită dimensiunii mari nu se instalează automat în distribuțiile de Linux. Pentru a instala Emacs, în linia de comandă, scrieți:

```
$ sudo apt-get install emacs21
Reading package lists... Done
Building dependency tree... Done
The following extra packages will be installed:
  emacs21-bin-common emacs21-common emacs21-el emacs21-nox libungif4g xaw3dg
Suggested packages:
  emacs21-el
The following NEW packages will be installed:
  emacs21 emacs21-bin-common emacs21-common emacs21-nox libungif4g xaw3dg
0 upgraded, 6 newly installed, 0 to remove and 0 not upgraded.
Need to get 13.3MB of archives.
After unpacking 44.2MB of additional disk space will be used.
Do you want to continue [Y/n]? y
Get:1 http://ro.archive.ubuntu.com dapper/main emacs21-common 1.4.17 [17.6kB]
[...]
Wrote /usr/share/emacs21/site-lisp/debian-startup.elc
Done
```

Un mare avantaj al editoarelor vim și emacs este faptul că se pot adapta la necesitățile utilizatorului. Astfel pe Internet există o multitudine de plug-in-uri și dicționare pentru acestea. (dicționare, aplicații, macro-uri, etc)

Important! Atunci când folosiți LiveCD s-ar putea să meargă foarte încet instalarea pachetelor. Cauza este lipsa de memorie RAM disponibilă. Astfel, se recomandă ca înainte de instalare să închideți mediul grafic cu una dintre comenzile:

```
$ sudo killall kdm
pentru KDM (Kubuntu) sau
$ sudo killall gdm
pentru Gnome (Ubuntu)
```

2. Compilare si Rulare Programe C

2.1. Introducere

Pasii care trebuie urmati pentru a compila un program C pe o distributie proaspat instalata de Linux (sau pe Live CD) sunt:

1. scriere cod sursa C (in nano, vim sau emacs)
2. instalare gcc
3. compilare cu gcc
4. rulare si verificare program

In aceasta parte a laboratorului ne propunem sa scriem un program C care sa adune numere intregi introduse de la tastatura. Ne dorim ca numarul de numere pe care le introducem sa nu fie fixat, asa ca vom folosi numarul 0 pentru a indica sfarsitul sirului de numere care trebuie adunate:

```
$ ./adun
Program de adunare de intregi

Dati numarul al 1-lea: 1
Dati numarul al 2-lea: 2
Dati numarul al 3-lea: 3
Dati numarul al 4-lea: 5
Dati numarul al 5-lea: 0
Rezultatul adunarii celor 4 numere este: 11

Multumim ca ati utilizat programul nostru!
```

2.2. Instalare pachete compilator gcc/g++

Inainte de a putea compila programe C/C++ cu gcc respectiv g++ trebuie sa instalam pachetele care le corespund. In afara de pachetele gcc si g++, vom mai instala urmatoarele pachete:

- libc6-dev
- binutils
- file
- make

pentru instalare trebuie sa lansam comanda:

```
$ apt-get install gcc g++ libc6-dev binutils file make
```

Numele apt vine de la Advanced Package Tool si este programul cu ajutorul caruia se instaleaza toate programele din Linux. Interfata grafica a acestuia este Synaptic Package Installer. Cand lansati comanda de mai sus, apt-get va descarca (download) de pe Internet pachetele pe care le-ati cerut si le va instala automat.

Important! Atunci cand folositi LiveCD s-ar putea sa mearga foarte incet instalarea pachetelor. Cauza este lipsa de memorie RAM disponibila. Astfel, se recomanda ca inainte de instalare sa inchideti mediul grafic cu una dintre comenzile:

```
$ sudo killall kdm
pentru KDM (Kubuntu) sau
$ sudo killall gdm
pentru Gnome (Ubuntu)
```

3.3. Compilare cu gcc/g++ si rulare

Pentru a compila un program cu gcc/g++ se foloseste sintaxa:

```
$ gcc -Wall sursa.c -o program
$ g++ -Wall sursa.cpp -o program
```

unde optiunea **-o** anunta compilatorul ca trebuie sa faca si legaturile intre program si bibliotecile pe care acesta le foloseste. (libc6-dev) Optiunea **-Wall** obliga compilatorul sa afiseze toate anunturile. (-Wall=Warnings on ALL events)

Programul care rezulta nu va avea o extensie predefinita (cum era in Windows .exe) ci va rula pe baza atributului Executable al sau. Dupa compilare se poate rula programul:

```
$ gcc -Wall adun.c -o adun
$ ./adun
Program de adunare de intregi
[...]
```

3.4. Makefile

3.4.1. Introducere

Make este un utilitar care utilizeaza un script, numit Makefile, pentru a determina automat secventa de pasi ce trebuie repetati deoarece niste fisiere s-au modificat. In principal este utilizat pentru:

- recompilarea programelor compuse din mai multe fisiere
- testarea programelor.

3.4.2. Formatul unui Makefile

Instructiunile pentru make se gasesc intr-un Makefile/makefile si determina ce actiuni trebuie facute pentru a satisface anumite cerinte. Sintaxa unui makefile este de forma:

```
Target... : dependencies ...
<tab>command
<tab>command
<tab>...
```

unde:

- target = numele unui fisier ce trebuie generat sau numele unei actiuni
- dependencies = lista de actiuni (fisiere) ce trebuie indeplinite (ce trebuie sa existe)

pentru a se realiza target-ul – make determina daca trebuie re-executat target-ul daca una dintre dependete s-a modificat (unul din fisierele din lista de dependente s-a modificat)

- `command` = comanda ce duce la realizarea target-ului (de obicei la comenzi sunt trecute comenzile de compilare care duc la realizarea target-ului)
- `<tab>` = caracterul tab

Regulile formeaza un lant de dependente. Pentru ca un target sa fie facut, se verifica daca toate dependentele lui sunt facute. Daca nu este indeplinita aceasta conditie, se executa dependentele intr-un fel asemanator.

Iata exemplu simplu de Makefile:

```
pregatire: mancare curatenie
    echo "Putem primi musafiri"
    touch pregatire
mancare: cumparaturi
    echo "Gata MANCARE"
    touch mancare
curatenie:
    echo "Gata CURATENIE"
    touch curatenie
cumparaturi:
    echo "Gata CUMPARATURI"
    touch cumparaturi
pierde_cumparaturi:
    rm cumparaturi
    echo "pierdut CUMPARATURI"
clean:
    rm cumparaturi
    rm curatenie
    rm mancare
    rm pregateste_musafiri
    echo "Gata ANULARE"
```

In Makefile se pot utiliza si variabile:

```
gccopt=-Wall
ccompiler=gcc
executab=adun
# daca sursa are mai multe fisiere, se pun cu spatiu:
# surse=adun.c matem.c ...
surse=adun.c

progc:
    $(ccompiler) $(gccopt) $(surse) -o $(executab)

clean:
    rm progc
```

3. Exerciții de laborator

Rezultatul final al acestui laborator va fi un program functional care sa adune numere si un script Makefile care sa poata instala programul in /usr/bin/ pentru a permite pornirea facila a acestuia.

1. Primul pas este parcurgerea tutorial-ului de vim. Exersand pe cerintele din acest tutorial veti ajunge foarte repede sa va obisnuiti cu vim, facand astfel foarte usor restul laboratorului. Pentru a porni tutorial-ul de vim lansati comanda:

```
$ vimtutor
```

2. Creati directorul in care veti lucra si porniti in el vim cu fisierul adun.c: (nu uitati de **:syntax on** in vim)

```
ubuntu@ubuntu:~$ mkdir adun
ubuntu@ubuntu:~$ cd adun
ubuntu@ubuntu:~/adun$ vim adun.c
si apoi introduceti urmatorul cod c:

#include <stdio.h>

int main()
{
    int i;
    long temp, result;

    i=1;           // cate numere am adunat
    temp=0;        // ultimul numar introdus
    result=0;      // suma numerelor introduse pana acum

    printf("Program de adunare de intregi\n\n");

    while(1)
    {
        printf("Dati numarul al %d-lea: ",i);
        scanf("%d",&temp);
        result=temp+result;

        if(temp==0)
        {
            printf("Rezultatul adunarii celor %d numere este: ",i-1);
            printf("%d\n\nMultumim ca ati utilizat ",result);
            printf("programul nostru!\n\n");
        }
        return 0;
        i++;
    }

    //nu ar trebui sa se ajunga aici ==> eroare
    return 1;
}
```

3. Instalati pachetele compilatorului, compilati programul si incercati sa il rulati. Vetii observa doua probleme:

1. programul nu porneste cand sa da numele lui ci doar cand se specifica si calea (./adun)
2. programul nu functioneaza corect

```
ubuntu@ubuntu:~/adun$ adun
bash: adun: command not found
ubuntu@ubuntu:~/adun$ ./adun
Program de adunare de intregi

Dati numarul al 1-lea: 12
ubuntu@ubuntu:~/adun$
```

4. Corectati programul C. Recompilati-l. Functioneaza corect acum?
5. Dupa ce programul se compileaza si functioneaza corect vom scrie prima parte din makefile:

1. porniti vim Makefile
2. introduceti urmatoarea intrare:

```
default: clean
        gcc -Wall adun.c -o adun

clean:
        rm -f adun

run: default
        ./adun
```

3. in linia de comanda porniti make:

```
ubuntu@ubuntu:~/adun$ make
rm adun
gcc -Wall adun.c -o adun
ubuntu@ubuntu:~/adun$ make run
rm adun
gcc -Wall adun.c -o adun
./adun
Program de adunare de intregi
[...]
```

4. incercati sa editati fisierul adun.c in vim si sa lansati make din comanda vim, dupa ce salvati ultimele schimbari. (:w si apoi **!make**) Astfel devine mult mai usoara programarea, inlesnind verificarea continua a sintaxei si a functionalitatii programului. *Aceste functii devin cu adevarat indispensabile in momentul in care programul la care lucrați depaseste nivelul unui singur fisier sursa si trebuie impartit pe module.*
6. Sa presupunem ca *adun* a devenit un program complex, pe care doriti sa il distribuiti la mai multe persoane si doriti sa aiba o instalare facila. Aceasta se poate face cu un script Makefile. Incercati sa adaugati la Makefile-ul pe care l-ati scris si tinteile install si uninstall.

Evident `install` va trebui sa aiba ca dependenta pe `uninstall` si pe `default`, ca sa se asigure ca versiunea veche este stearsa si ca se instaleaza ultima versiune. Scrieti in `Makefile` comanda de consola care copiaza executabilul **adun** in **/usr/bin**. (acolo se afla si **gcc** si **g++**) Scrieti si comanda care sterge fisierul **/usr/bin/adun**.

Porniti **make install** din linia de comanda si vedeti ce se intampla. S-ar putea cumva sa nu aveti drepturi de scriere in `/usr/bin` ca utilizator normal? Mergeti in **/etc** si incercati sa porniti **adun**. Ar trebui sa porneasca fara probleme. Puteti explica de ce?

```
ubuntu@ubuntu:/etc$ adun
Program de adunare de intregi
[...]
```

7. Pentru a impiedica eco-ul unei comenzi la ecran se poate pune un **@** inaintea acesteia. Incercati sa introduceti simbolul **@** inaintea liniilor din tintele *default*, *clean* si *run*. Rulati **make run**. Incercati acum sa adaugati dupa linia `gcc -o ...` linia **@echo Compile ready...** Cum apare acum? Cosmetizati intreg `Makefile`-ul incat sa apara mesaje corecte la instalare si la deinstalare.
8. Acum puteti considera ca aveti o varinta foarte simpla de pachet care se poate distribui. Faceti o arhiva `tar.bz2` cu sursa si `Makefile`-ul. Aceasta puteti sa o distribuiti prietenilor.